# Kamino Redeem-in-Kind

Security Assessment

March 19th, 2026 — Prepared by OtterSec


Ajay Shankar Kunapareddy d1r3wolf@osec.io

Xiang Yin soreatu@osec.io

Tamta Topuria tamta@osec.io

# Table of Contents

**Executive Summary** **2**

Overview 2

Key Findings 2

**Scope** **3**

**General Findings** **4**

OS-KRK-SUG-00 | Ambiguous Penalty Variable Naming 5

## Appendices

**Vulnerability Rating Scale** **6**

**Procedure** **7**

© 2026 Otter Audits LLC. All Rights Reserved.

# 01 — Executive Summary

## Overview

Kamino Finance engaged OtterSec to assess the `redeem-in-kind` feature. This assessment was conducted between March 15th and March 17th, 2026. For more information on our auditing methodology, refer to Appendix B.

## Key Findings

We produced 1 finding throughout this audit engagement.

In particular, we suggested utilizing separate names for the initial and recomputed penalties to ensure semantic clarity and avoid any ambiguity (OS-KRK-SUG-00).

© 2026 Otter Audits LLC. All Rights Reserved.

# 02 — Scope

The source code was delivered to us in a Git repository at https://github.com/Kamino-Finance/kvault-private. The audit was performed against PR#59.

**A brief description of the program is as follows:**

| Name | Description |
|---|---|
| redeem-in-kind | Allows a user to redeem vault shares directly for a selected reserve's cTokens instead of withdrawing idle underlying liquidity. It computes the user's pro-rata entitlement and withdrawal penalty, burns the corresponding shares, reduces the vault's reserve allocation, and transfers the reserve cTokens to the user. |

© 2026 Otter Audits LLC. All Rights Reserved.

# 03 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

| ID | Description |
|---|---|
| OS-KRK-SUG-00 | `redeem_in_kind` re-utilizes `withdrawal_penalty_f` for two distinct penalty calculations, which obscures that the second value is a meaningful recomputation rather than the same quantity carried forward. |

© 2026 Otter Audits LLC. All Rights Reserved.

## Ambiguous Penalty Variable Naming OS-KRK-SUG-00

### Description

`redeem_in_kind` computes two different penalties, but both are stored in a variable named `withdrawal_penalty_f`. The first `withdrawal_penalty_f` corresponds to the initial penalty derived from the user's theoretical withdrawal amount before converting into `cTokens` and before applying the cap imposed by the vault's actual reserve allocation. The second `withdrawal_penalty_f` is a newly recomputed penalty based on the actual liquidity value represented by the final `cToken` amount. Thus, keeping the same identifier can obscure the intended implementation, rendering the two-stage pricing flow harder to follow.

### Remediation

Utilize separate names for the initial and recomputed penalties, such as `initial_withdrawal_penalty_f` and `recomputed_withdrawal_penalty_f`, respectively, to ensure semantic clarity and avoid any ambiguity.

© 2026 Otter Audits LLC. All Rights Reserved.

# A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

**CRITICAL**

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

**HIGH**

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

**MEDIUM**

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

**LOW**

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

**INFO**

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2026 Otter Audits LLC. All Rights Reserved.

# B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.